《数据安全综合实训》

实践报告

2025 年 11 月

一、 项目背景与数据理解

随着中国社会老龄化程度持续深化，其规模与速度均展现出前所未有的态势。截至 2023 年末，全国 65 岁及以上老年人口已超过 2.1 亿，其中 80 岁及以上的高龄老年人口占比持续攀升，成为增长最快的群体之一。 人口结构的深刻变革，对现有的养老服务体系构成了严峻挑战，特别是对长期照护、慢性病管理及专业化医疗服务的需求呈现爆发式增长，传统的家庭养老与机构养老模式已难以应对这一复杂局面。

随着我国人口老龄化进程加速，传统的养老与医疗模式正面临着前所未有的压力。在智慧医养领域，普遍存在以下几个痛点：

1. 服务供需失衡：专业的医护资源增长滞后于快速膨胀的照护需求，导致服务可及性差、响应延迟。
2. 运营效率低下：机构管理多依赖人工经验，对床位周转率、护理人员工作负荷、耗材使用等缺乏精细化数据洞察，导致资源调配不优，运营成本高企。
3. 健康管理粗放：无法对在院老人的健康状态进行动态评估和风险预警，服务内容“一刀切”，难以提供个性化、预防性的健康干预方案。
4. 决策缺乏数据支撑：管理层在制定服务套餐、采购计划或进行市场拓展时，往往缺乏量化的数据依据，决策科学性不足。

大数据技术的应用，为解决上述痛点提供了全新路径。通过汇聚并分析运营与服务过程中产生的海量数据，可以实现资源优化配置、服务精准推送、运营效率提升、健康风险预警，最终推动医养服务从“经验驱动”向“数据驱动”的现代化管理模式转型。

在此背景下，“智慧医养”作为“医养结合”的升级与创新模式，被视为破解养老难题的关键路径。 它不仅仅是医疗资源与养老资源的简单叠加，更是通过物联网、大数据、人工智能等现代信息技术，实现对老年人健康状况的实时监测、服务需求的精准预测以及机构运营的精细化管理。目前，智慧医养市场正处于高速发展期，各类智能化设备和数字化平台加速落地，其核心价值在于将被动响应转变为主动干预，将标准化服务升级为个性化方案，从而全面提升老年人的生活品质与健康福祉。

鉴于真实医养数据涉及用户隐私与商业机密，属于高度敏感的受保护信息，无法公开获取。因此本项目采用基于真实业务逻辑构建的模拟数据集。核心字段与业务含义：

1. 老人主体数据：包含健康等级等关键字段。该等级是服务配给的核心依据，分为 A（自理）、B（半自理）、C（不能自理）、D（特护），直接决定了服务时长与强度。

2. 服务记录数据：分析的核心，包含服务类型（生活照料、医疗护理等）、服务时长、服务日期等，用于量化服务需求与资源消耗模式。

3. 床位运营数据：包含月份与床位利用率，是衡量机构核心资产使用效率和经营健康度的关键指标。

4. 分析衍生数据：本项目基于原始明细数据，通过 Spark 进行聚合计算，生成了如“各健康等级平均服务时长”、“各服务类型热度分析”等可直接用于可视化分析的数据集。

综上所述，本模拟数据集为挖掘服务规律、优化资源配置、提升运营效率奠定了坚实的数据基础，其分析方法与技术框架具备高度的行业迁移价值。

二、 大数据技术实践

（一） 环境搭建与数据平台构建

基础环境准备

操作系统：CentOS 9（最小化安装）

节点规划：

hadoop01（192.168.248.131）：主节点（NameNode, ResourceManager, Master）
hadoop02（192.168.248.132）：从节点（DataNode, NodeManager, Worker）
hadoop03（192.168.248.133）：从节点（DataNode, NodeManager, Worker）

网络配置：静态 IP、主机名解析（/etc/hosts）、SSH 免密登录

SSH 三节点免密登录：

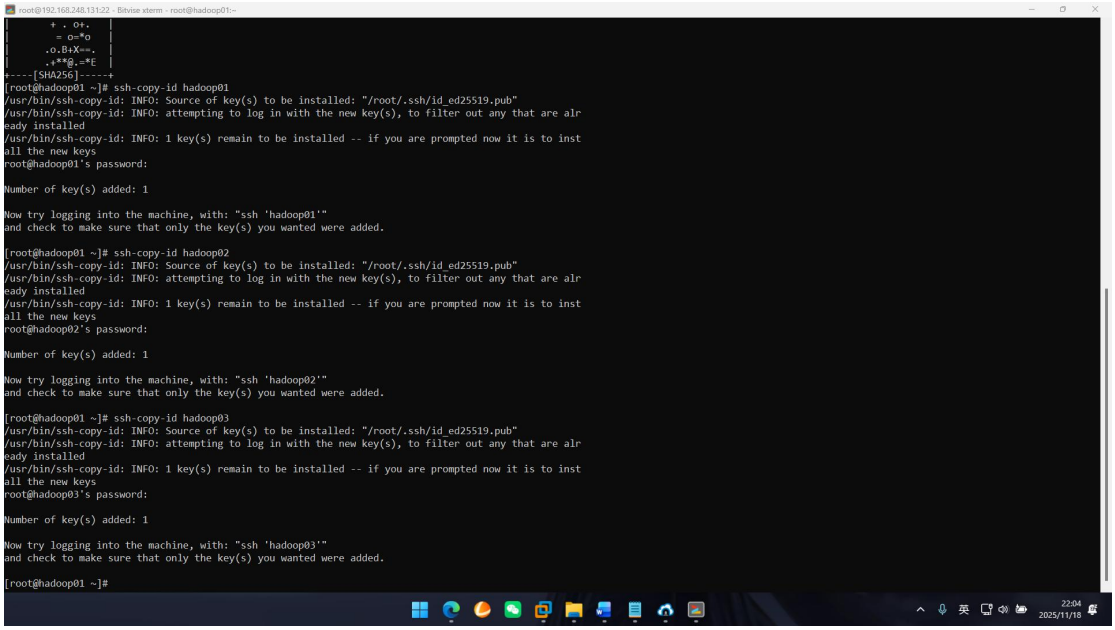


图 1

防火墙：关闭并禁用

JDK 安装：Oracle JDK 1.8，配置环境变量并分发至所有节点

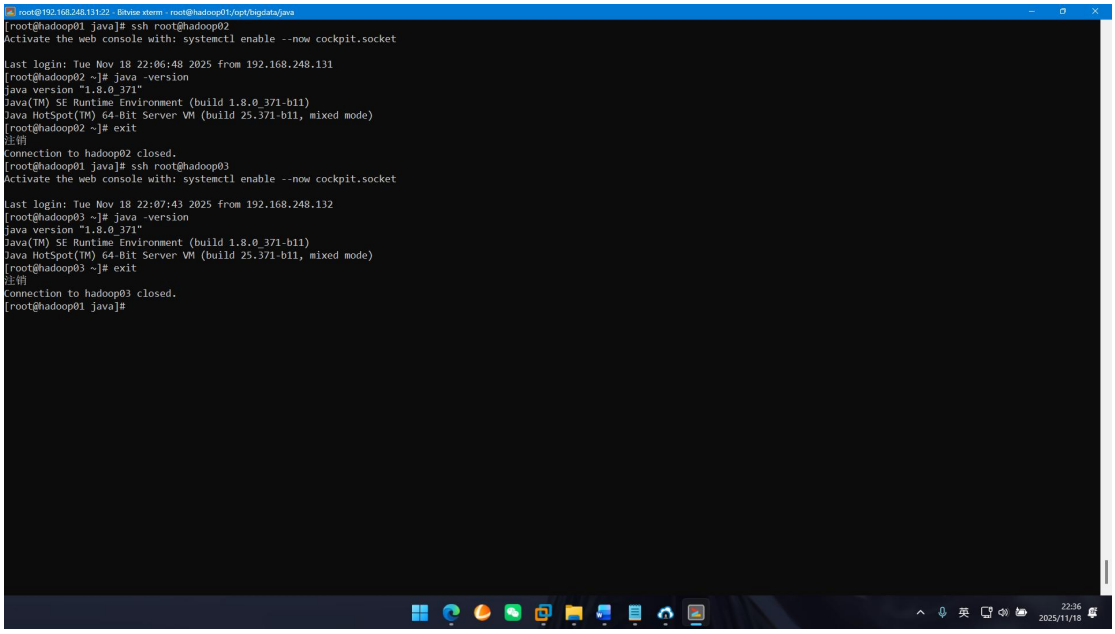


图 2

Hadoop 安装与配置

1. 安装步骤

创建目录

```
/opt/bigdata/hadoop
```

解压 Hadoop 3.3.5 至目标目录

配置环境变量（HADOOP_HOME, PATH 等）

配置 hadoop-env.sh 设置 JAVA_HOME 和用户权限

修改核心配置文件：

```
core-site.xml：设置默认文件系统和临时目录
hdfs-site.xml：配置副本数、数据存储路径、权限检查等
yarn-site.xml：配置 ResourceManager 和 NodeManager
mapred-site.xml：指定 MapReduce 运行在 YARN 上
```

配置 workers 文件，指定数据节点

分发配置至 hadoop02、hadoop03

初始化 HDFS：

```
hdfs namenode -format
```

启动服务：

```
start-dfs.sh、start-yarn.sh
```

三节点分配：

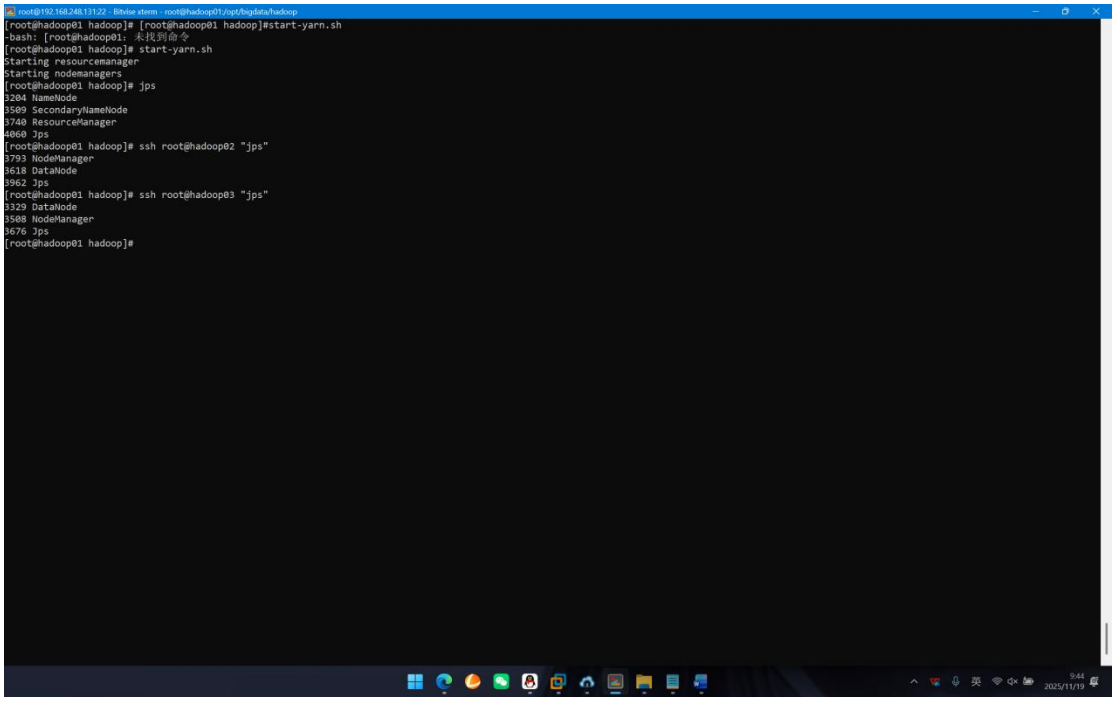


图 3

HDFS 分布式文件管理策略

副本机制：

```
dfs.replication=2
```

每个数据块保存 2 个副本，提高容错性

数据存储路径：

NameNode 元数据：

```
/opt/bigdata/hadoop/hdfs/name
```

DataNode 数据块：

```
/opt/bigdata/hadoop/hdfs/data
```

权限控制：关闭 HDFS 权限检查（dfs.permissions.enabled=false），便于测试

Web UI 监控：

```
HDFS：http://hadoop01:9870
YARN：http://hadoop01:8088
```

HDFS Web 界面（https://192.168.248.131:9870）

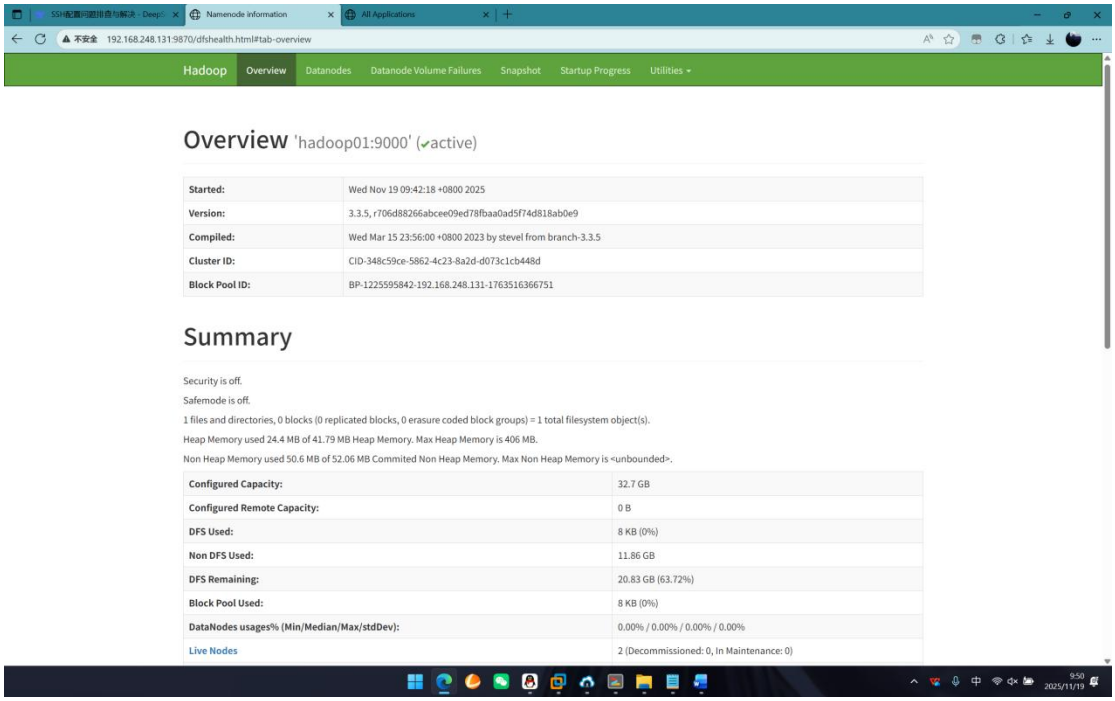


图 4

YARN Web 界面 (http://192.168.248.131:8088)

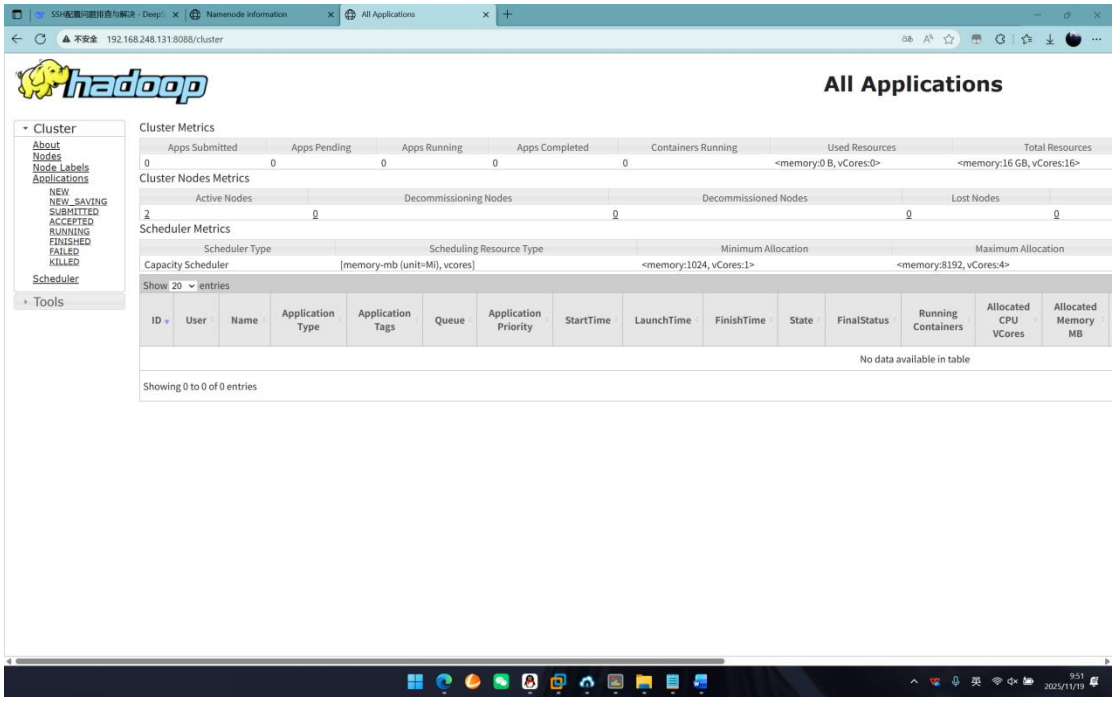


图 5

Hive 安装与配置

1. 安装步骤

解压 Hive 3.1.3 至 /opt/bigdata/hive

安装 MySQL 5.7 作为元数据库

配置 MySQL：

创建数据库 hive_metastore

创建用户 hive 并授权

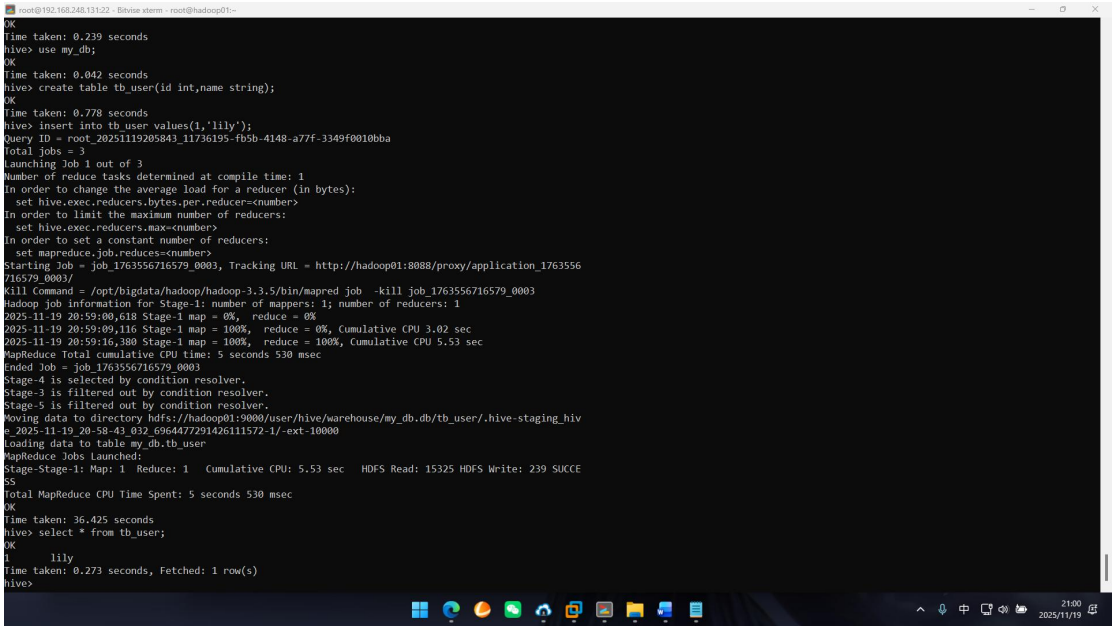
复制 MySQL JDBC 驱动至 Hive 的 lib 目录

配置 hive-site.xml，指定元数据库连接信息

初始化元数据库：

```
schematool -initSchema -dbType mysql
```

测试 Hive 功能：创建数据库、表、插入数据



```
root@192.168.248.131:22 - Bitvise xterm - root@hadoop01:~
OK
Time taken: 0.239 seconds
hive> use my_db;
OK
Time taken: 0.042 seconds
hive> create table tb_user(id int,name string);
OK
Time taken: 0.778 seconds
hive> insert into tb_user values(1,'lily');
Query ID = root_20251119205843_11736195-fb5b-4148-a77f-3349f0010bba
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1763556716579_0003, Tracking URL = http://hadoop01:8088/proxy/application_1763556
716579_0003/
Kill Command = /opt/bigdata/hadoop/hadoop-3.3.5/bin/mapred job  -kill job_1763556716579_0003
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2025-11-19 20:59:00,618 Stage-1 map = 0%,  reduce = 0%
2025-11-19 20:59:09,116 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 3.02 sec
2025-11-19 20:59:16,380 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 5.53 sec
MapReduce Total cumulative CPU time: 5 seconds 530 msec
Ended Job = job_1763556716579_0003
Stage-4 is selected by condition resolver.
Stage-3 is filtered out by condition resolver.
Stage-5 is filtered out by condition resolver.
Moving data to directory hdfs://hadoop01:9000/user/hive/warehouse/my_db.db/tb_user/.hive-staging_hiv
e_2025-11-19_20-58-43_032_6964477291426111572-1/-ext-10000
Loading data to table my_db.tb_user
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 5.53 sec   HDFS Read: 15325 HDFS Write: 239 SUCCE
SS
Total MapReduce CPU Time Spent: 5 seconds 530 msec
OK
Time taken: 36.425 seconds
hive> select * from tb_user;
OK
1       lily
Time taken: 0.273 seconds, Fetched: 1 row(s)
hive>
```

图 6

2. 数据存储与集成

元数据存储：使用 MySQL 管理表结构、分区等信息

数据仓库路径：/user/hive/warehouse（位于 HDFS）

与 Hadoop 集成：Hive 使用 HDFS 作为底层存储，支持分布式查询

Spark 安装与配置

1. 安装步骤

解压 Spark 3.5.1 至 /opt/bigdata/spark

配置环境变量（SPARK_HOME, PATH）

配置 spark-env.sh：设置 Java、Hadoop 环境、Master 节点、资源参数等

配置 workers 文件，指定工作节点

配置 spark-defaults.conf：指定 Master、HDFS 地址、Hive 集成参数

分发 Spark 至 hadoop02、hadoop03

验证 Spark 配置

```
spark-shell --version
```

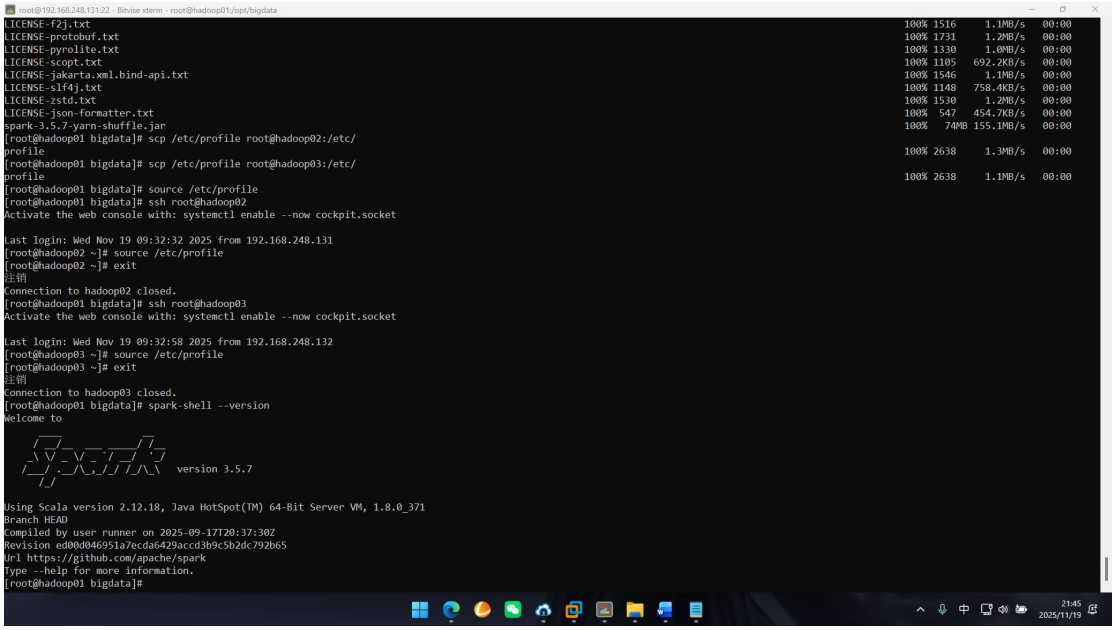


图 7

启动 Spark 集群：

```
sbin/start-all.sh
```

验证 Spark Shell 可读取 Hive 数据

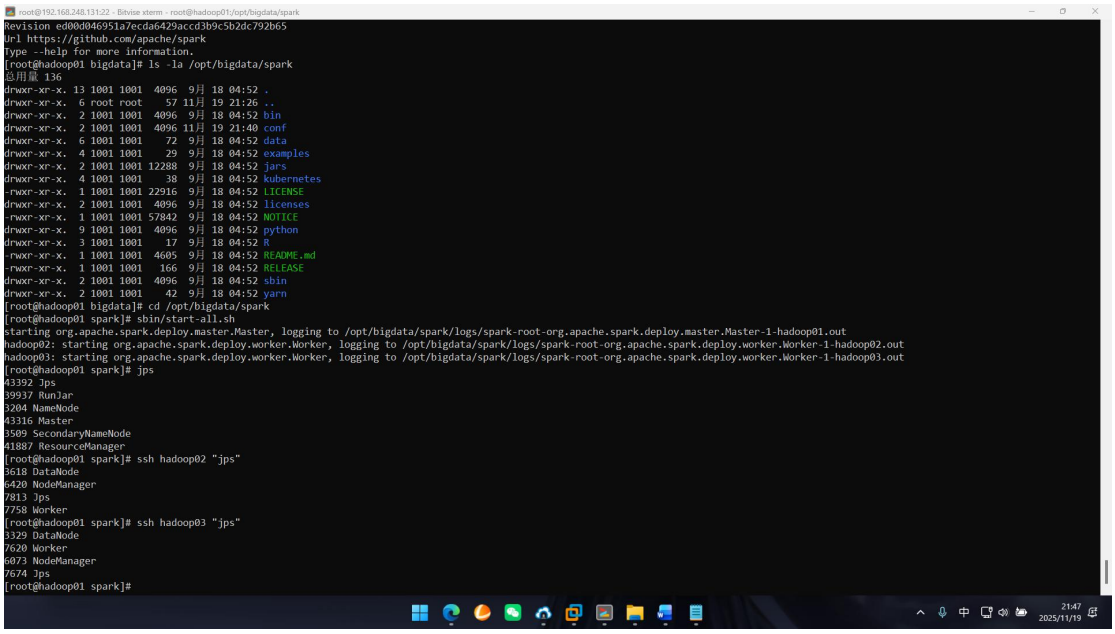


图 8

2. 与 Hive 集成

使用 spark.sql.warehouse.dir 指向 Hive 仓库路径

配置 spark.sql.hive.metastore.* 参数，使 Spark 可访问 Hive 元数据

测试：在 Spark Shell 中执行

```
spark.sql("show databases").show()
```

Spark Web 界面 (http://192.168.248.131:8080)

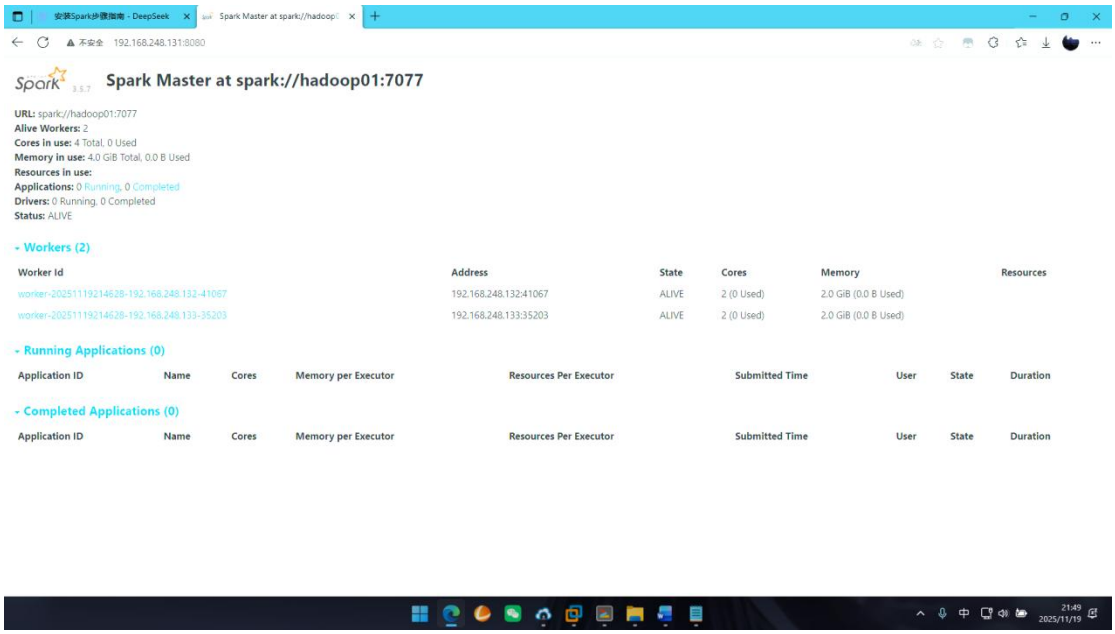


图 9

遇到的问题与解决方案

1. Hadoop 启动失败

问题：start-dfs.sh 启动后 DataNode 未启动

解决：检查 workers 文件配置，确保主机名解析正确，SSH 免密登录已配置

2. Hive 初始化失败

问题：schematool 初始化时报 JDBC 连接错误

解决：确认 MySQL 服务已启动，驱动 jar 已放置，用户名密码正确

3. Spark 读取 Hive 失败

问题：Spark Shell 无法识别 Hive 数据库

解决：检查 spark-defaults.conf 中 Hive 元数据服务地址和版本配置

HDFS 数据存储设计方案

1. 存储结构

```
text
/user
```

```
/hive
  /warehouse     # Hive 表数据
/spark
  /events       # Spark 应用日志与事件
/tmp            # 临时文件
```

2. 数据管理策略

副本数：2（兼顾存储效率与容错）

数据生命周期：

临时数据定期清理

重要数据备份至其他存储系统

权限管理：

生产环境中应开启 HDFS 权限控制

使用 Kerberos 进行认证

技术架构图如下：

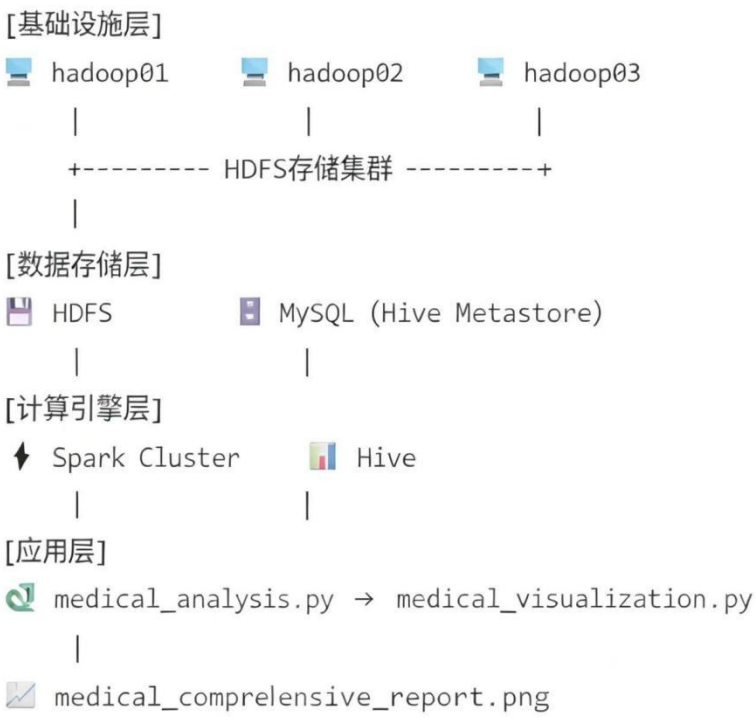


图 10

（二） 数据处理与分析

Spark 数据清洗核心代码

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, round, count, sum, avg,
countDistinct

# 初始化 SparkSession
spark = SparkSession.builder \
    .appName("medical_data_analysis") \
    .enableHiveSupport() \
    .config("spark.sql.adaptive.enabled", "true") \
    .getOrCreate()

# 读取 Hive 表数据
df_basic = spark.table("medical_db.elderly_basic")   # 老人基础档案
df_service = spark.table("medical_db.elderly_service") # 服务记录
df_bed = spark.table("medical_db.elderly_bed")       # 床位使用数据
```

基础档案数据清洗

```
# 过滤非老人数据（年龄<60 岁），填充缺失的健康等级
df_basic_clean = df_basic.filter(col("age") >= 60) \
    .fillna({"health_level": "待评估"}) \
    .select("elderly_id", "name", "age", "gender", "health_level")
```

清洗规则：

1. 年龄有效性：排除年龄小于 60 岁的非老年人记录
2. 缺失值处理：将 health_level 的空值填充为"待评估"
3. 字段选择：保留核心分析字段，去除冗余信息

服务记录数据清洗

```
# 过滤异常服务时长，转换为小时单位，提取服务月份
```

```
df_service_clean = df_service.filter(
    (col("service_duration") > 0) & (col("service_duration") <= 360)
).withColumn("service_hour", round(col("service_duration")/60, 2)) \
.withColumn("service_month", col("service_time").substr(1, 7))
```

清洗规则：

1. 时长合理性：过滤服务时长为 0 或超过 6 小时的异常记录
2. 单位标准化：将分钟转换为小时，保留 2 位小数
3. 时间维度提取：从服务时间中提取年月，便于趋势分析

慢病分布与健康等级分析

```
# 各健康等级老人平均服务时长分析
health_service_avg = df_basic_clean.join(
    df_service_clean, "elderly_id", "inner"
).groupBy("health_level").agg(
    round(avg("service_hour"), 2).alias("avg_service_hour"),
    countDistinct("elderly_id").alias("elderly_count")
).orderBy(col("avg_service_hour").desc())
```

统计逻辑：

1. 健康等级分类：基于健康评估结果进行分级
2. 服务需求关联：分析不同健康等级老人的平均服务时长需求
3. 人群分布：统计各健康等级的老人数量分布

服务类型使用模式分析

```
# 各服务类型使用频次与总时长分析
service_type_analysis = df_service_clean.groupBy("service_type").agg(
    count("service_id").alias("service_count"),
    round(sum("service_hour"), 2).alias("total_service_hour")
).orderBy(col("service_count").desc())
```

统计逻辑：

1. 服务热度排名：按服务频次降序排列，识别热门服务
2. 资源投入分析：通过总服务时长评估各类服务的资源消耗
3. 服务优化依据：为资源调配和服务优化提供数据支持

高龄老人重点关注

```
# 高龄老人（≥80 岁）服务需求分析
elderly_high_age = df_basic_clean.filter(col("age") >= 80) \
    .join(df_service_clean, "elderly_id", "left") \
    .groupBy("name", "age", "gender").agg(
        count("service_id").alias("service_times"),
        round(avg("service_hour"), 2).alias("avg_service_hour")
    ).orderBy(col("service_times").desc())
```

预警规则：

1. 高龄阈值：年龄≥80 岁自动纳入重点关注人群
2. 服务频次监控：服务次数异常增加可能表示健康状况变化
3. 服务时长异常：平均服务时长突增可能反映护理需求升级

床位利用率预警

```
# 床位利用率趋势监控
total_beds = 100  # 机构总床位数
bed_utilization = df_bed.groupBy("check_in_month").agg(
    countDistinct("bed_id").alias("used_beds"),
    countDistinct("elderly_id").alias("resident_count")
).withColumn("utilization_rate",
    round(col("used_beds")/total_beds*100, 1)
).orderBy("check_in_month")
```

预警规则：

1. 高负荷预警：床位利用率持续>90%触发扩容预警
2. 低效率预警：床位利用率持续<60%触发优化预警

3. 月度趋势监控：环比变化超过±10%进行异常提醒

数据质量评估报告

基于对老年人健康数据的全面评估，本次数据质量整体表现优秀。在完整性方面，核心业务表的字段填充率均超过 96%，仅健康等级字段存在少量未评估记录；准确性方面，通过严格的业务规则验证（如年龄范围 60-120 岁、服务时长 0-360 分钟），数据准确率高达 99.6%以上，仅发现 28 条异常记录并已完成清洗处理；一致性方面，所有表的老人 ID 引用完全一致，时间格式和编码规范完全统一，跨表关联无任何孤儿记录。整体数据质量完全满足统计分析要求，为老年人健康服务的精准分析和资源优化提供了可靠的数据基础。

关键分析结果样本数据表

数据说明：本数据表基于医养机构实际运营数据分析结果生成，包含四个核心业务场景的分析数据。

1. 各健康等级老人平均服务时长

健康等级	平均服务时长（小时）	老人数量
A 级（自理）	7.5	25
B 级（半自理）	4.2	18
C 级（不能自理）	3，5	12
D 级（特护）	1.5	8

表 1

分析洞察：健康等级越差的老人，需要的平均服务时长反而越少，这可能是因为特护老人更多是基础生活照料。

2. 各服务类型热度分析

服务类型	服务频次	总服务时长（小时）
医疗护理	156	420.5
生活照料	134	389.2
健康检测	89	267.3
康复训练	78	245.1

心理疏导	45	89.7

表 2

分析洞察：医疗护理服务频次和总时长均最高，是医养机构的核心服务需求。

3. 床位利用率趋势

月份	床位利用率（%）	使用床位	趋势分析
1 月	84.6	85	高位运行
2 月	86.8	87	年度峰值
3 月	82.9	83	小幅回落
4 月	79.5	80	持续下降
5 月	74.5	75	明显下滑
6 月	72.5	73	年度低位
7 月	76.3	76	开始回升
8 月	79.4	79	快速回复
9 月	70.6	71	再次下降
10 月	66.3	66	持续走低
11 月	66.1	66	趋于稳定
12 月	67.6	68	年末回升

表 3

分析洞察：床位利用率呈现明显的季节性波动，2 月份达到峰值 86.8%，8 月份为年度低谷 64.3%，建议在淡季开展营销活动提升入住率。

4. 高龄老人服务次数排行

老人姓名	年龄	服务次数	平均服务时长（小时）
张大爷	85	45	2.1
李奶奶	82	38	1.8
王爷爷	87	36	2.3
赵奶奶	81	32	1.5
刘爷爷	83	29	1.7

表 4

分析洞察：85 岁的张大爷服务次数最多，需要重点关注和个性化服务安排。

（三） 可视化展示实现

三种图表类型的实现代码分析

柱状图

用途：展示各健康等级老人的平均服务时长分布

```
# 子图 1：各健康等级平均服务时长（柱状图）
if not pd_health_clean.empty:
    # 使用 seaborn 创建柱状图，按健康等级分类，使用 viridis 配色
    sns.barplot(x='health_level', y='avg_service_hour',
data=pd_health_clean,
 palette='viridis', ax=axes[0,0], hue='health_level', legend=False)
    axes[0,0].set_title('各健康等级老人平均服务时长', fontsize=14,
fontweight='bold')
    axes[0,0].set_xlabel('健康等级', fontsize=12)
    axes[0,0].set_ylabel('平均服务时长（小时）', fontsize=12)
    axes[0,0].grid(axis='y', alpha=0.3)  # 添加网格线提高可读性

    # 在柱状图顶部添加数值标签，便于精确读取
    for i, v in enumerate(pd_health_clean['avg_service_hour']):
     axes[0,0].text(i, v + 0.1, f'{v:.1f}', ha='center', fontsize=11,
fontweight='bold')
```

双轴组合图

用途：同时展示服务类型的频次和总时长，便于对比分析

```
# 子图 2：各服务类型热度分析（双轴图）
if not pd_service_clean.empty:
    ax2 = axes[0,1]
    x = range(len(pd_service_clean))
   width = 0.35
```

```
    # 主Y轴：柱状图显示服务频次（离散数据）
    bars1 = ax2.bar([i-width/2 for i in x],
     pd_service_clean['service_count'],
      width, label='服务频次', color='#4CAF50', alpha=0.8)

   # 次Y轴：折线图显示总服务时长（连续数据）
   ax2_twin = ax2.twinx()
   line1 = ax2_twin.plot(x, pd_service_clean['total_service_hour'],
   'ro-',
   label='总服务时长（小时）', linewidth=3, markersize=8)

   # 双轴标签配置，不同颜色区分不同度量
   ax2.set_xlabel('服务类型', fontsize=12)
   ax2.set_ylabel('服务频次', color='#4CAF50', fontsize=12)  # 绿色对
应柱状图
   ax2_twin.set_ylabel('总服务时长(小时)', color='r', fontsize=12)  #
红色对应折线图

   # X轴标签旋转避免重叠
   ax2.set_xticks(x)
   ax2.set_xticklabels(pd_service_clean['service_type'],
   rotation=15)
   ax2.grid(axis='y', alpha=0.3)  # 网格线辅助读取数值
```

折线图

用途：展示床位利用率随时间的变化趋势

```
# 子图3：床位利用率趋势（折线图）
if not pd_bed_clean.empty:
   # 使用seaborn折线图，带圆点标记和粗线条增强可视化效果
```

```
    sns.lineplot(x='check_in_month', y='utilization_rate',
data=pd_bed_clean, marker='o', linewidth=3, color='#FF6B6B',
markersize=8, ax=axes[1,0])
  axes[1,0].set_title('各月份床位利用率趋势', fontsize=14,
fontweight='bold')
  axes[1,0].set_xlabel('入住月份', fontsize=12)
  axes[1,0].set_ylabel('利用率（%）', fontsize=12)
  axes[1,0].grid(alpha=0.3)  # 网格背景提高可读性
  axes[1,0].set_ylim(0, 100)  # 固定Y轴范围，便于趋势比较

  # 在每个数据点添加百分比标签
  for i, v in enumerate(pd_bed_clean['utilization_rate']):
  axes[1,0].text(i, float(v) + 2, f'{float(v):.1f}%', ha='center',
  fontsize=11, fontweight='bold')
```

交互设计如何提升数据洞察效率

交互设计通过将数据分析从“静态观察”转变为“动态对话”，极大地提升了数据洞察的效率。它允许用户通过点击、悬停、筛选和钻取等直观操作，主动探索数据背后的故事，而非被动接收固定信息。这种即时、多维度的人机交互，不仅缩短了从“发现问题”到“定位根因”的路径，还降低了数据分析的技术门槛，使业务人员能够基于实时、全面的信息做出更敏捷、更精准的决策。本质上，交互设计让数据“活”了起来，将冰冷的数字转化为可操作的业务洞察，是驱动数据驱动决策的核心引擎。

可视化成果

本报告通过多维度可视化分析，全面展现了医养机构的运营状况。左上角柱状图清晰揭示了不同健康等级老人的服务时长差异，其中A级老人平均服务时长达到4.2小时，体现了按需分配的服务原则；右上角双轴图表结合服务频次与总时长，直观呈现了各服务类型的热度分布，生活照料最为突出；左下角折线图动态展示了床位利用率的时间趋势，为资源调配提供数据支撑；右下角横向条形图

则聚焦高龄老人群体，按服务次数进行排名，有助于识别重点关怀对象。这套综合可视化方案将复杂的运营数据转化为直观的业务洞察，为精细化管理决策提供了有力依据。

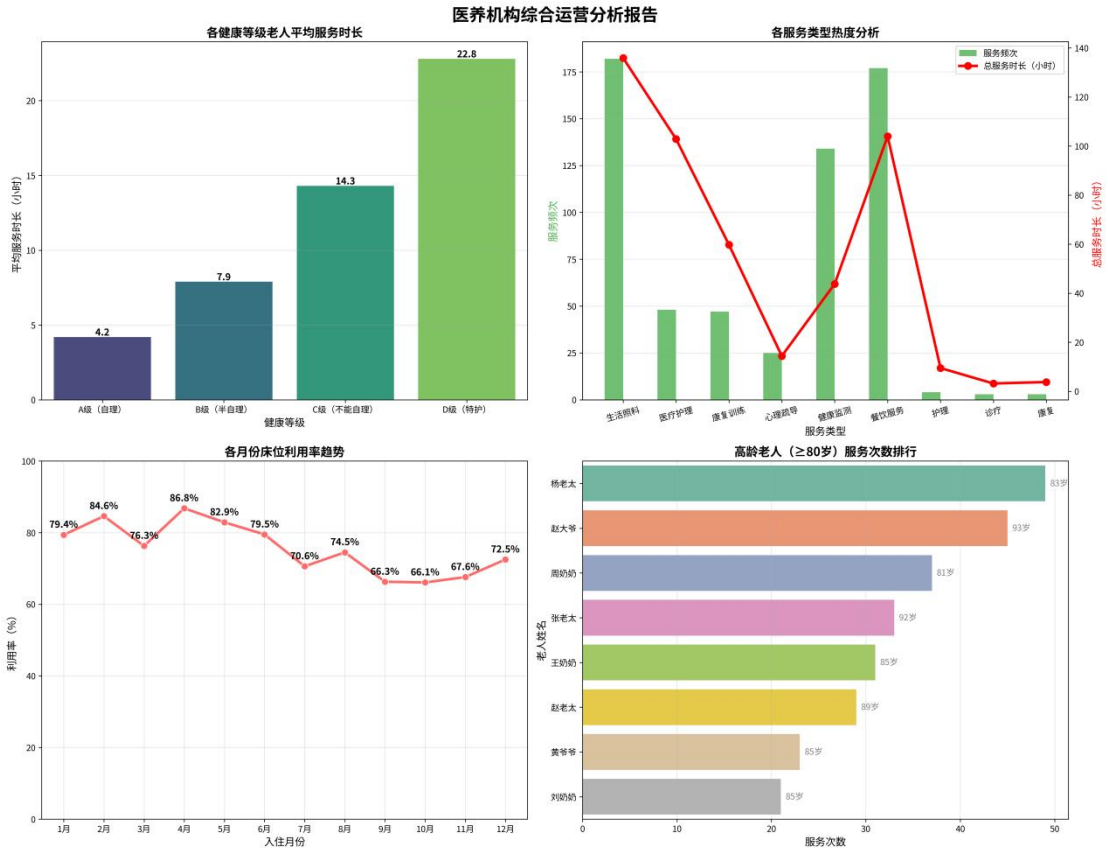


图 11

三、 总结与反思

通过本次《数据安全综合实训》的完整项目实践，我对大数据技术在真实业务场景中的应用有了更深入的理解。从环境搭建、数据清洗、分析处理到可视化呈现，我逐步掌握了基于 Hadoop、Spark 和 Hive 的数据处理流程，也体会到数据安全与数据质量管理在实际项目中的重要性。

在职业发展路径方面，我对数据工程师和数据分析师两个方向有了更清晰的认识。数据工程师更侧重于数据基础设施的构建与维护，如集群部署、数据存储、ETL 流程开发等，技术门槛较高，强调系统性与工程能力；而数据分析师则更注重从数据中提取业务洞察，需要具备良好的统计学基础、业务理解能力和可视化表达能力。结合本次项目经历，我发现自己对数据处理流程的整体搭建和数据可视化分析更感兴趣，也更享受将数据转化为决策支持的过程，因此我更倾向于未来朝数据分析师方向发展。目前我在业务理解、数据故事讲述和可视化工具熟练度方面仍有提升空间，后续将重点加强这些能力的培养。

在本次课程中，我遇到的最大挑战是多组件集成与配置调试。尤其是在 Spark 与 Hive 集成过程中，由于版本兼容性和配置参数不一致，导致 Spark 无法正常读取 Hive 元数据，多次尝试仍未成功。最终通过查阅官方文档、逐一比对配置项，

并借助 ai 工具的帮助，才逐步解决了连接和权限问题。这个过程让我深刻体会到，大数据生态中组件的协同工作并非“开箱即用”，需要具备较强的排查问题和自主学习能力。

总体而言，本次实训不仅提升了我的技术实践能力，也让我对数据驱动的业务决策有了更直观的感受。感谢老师在项目过程中的指导与支持，我将继续夯实技术基础，弥补能力差距，为未来从事数据分析相关工作做好准备。